In [ ]:
```
# PROJET 2 : PRÉDICTION DE LA CONCENTRATION EN MINERAI
# Dataset : Quality Prediction in a Mining Process (Kaggle)
# Objectif : Prédire % Silica Concentrate (impureté) à partir des variables géol
```

In [38]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import sklearn
from sklearn.model_selection import train_test_split, GridSearchCV, cross_val_sc
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.neural_network import MLPRegressor
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
from sklearn.pipeline import Pipeline
from sklearn.linear_model import Ridge
import warnings
warnings.filterwarnings('ignore')
```

In [3]:
```
# Style des graphiques
plt.style.use('seaborn-v0_8-whitegrid')
sns.set_palette("husl")
```

In [ ]:
```
# 1. CHARGEMENT DES DONNÉES
# ============================================================================

# Chemin du fichier (à adapter selon votre environnement)
# D'après vos captures, le fichier est sur le Desktop
donnees = pd.read_csv("C:/Users/Windows/Desktop/MiningProcess_Flotation_Plant_Da
```

In [1]:
```
print('PRESENTATION DES DONNEES')
```

```
PRESENTATION DES DONNEES
```

In [ ]:
```

```

In [36]:
```
donnees.head()
```

Out[36]:

| | date | % Iron Feed | % Silica Feed | Starch Flow | Amina Flow | Ore Pulp Flow | Ore Pulp pH | Ore Pulp Density | Flotation Column 01 Air Flow | Flotatic Colum 02 A Flo |
|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 2017-03-10 01:00:00 | 55,2 | 16,98 | 3019,53 | 557,434 | 395,713 | 10,0664 | 1,74 | 249,214 | 253,2 |
| **1** | 2017-03-10 01:00:00 | 55,2 | 16,98 | 3024,41 | 563,965 | 397,383 | 10,0672 | 1,74 | 249,719 | 250,5 |
| **2** | 2017-03-10 01:00:00 | 55,2 | 16,98 | 3043,46 | 568,054 | 399,668 | 10,068 | 1,74 | 249,741 | 247,8 |
| **3** | 2017-03-10 01:00:00 | 55,2 | 16,98 | 3047,36 | 568,665 | 397,939 | 10,0689 | 1,74 | 249,917 | 254,4 |
| **4** | 2017-03-10 01:00:00 | 55,2 | 16,98 | 3033,69 | 558,167 | 400,254 | 10,0697 | 1,74 | 250,203 | 252,1 |

5 rows × 24 columns

In [7]:
```
# Nombre de Lignes(data points) et des colonnes (Features- variables )
print('taille de la de donnees: ',donnees.shape)
print('Nombres de lignes:', donnees.shape[0])
print('Nombres de Colonnes:', donnees.shape[1])
```

```
taille de la de donnees:  (737453, 24)
Nombres de lignes: 737453
Nombres de Colonnes: 24
```

In [2]:
```
print('INFORMATIONS SUR LES TYPES DE DONNEES')
```

```
INFORMATIONS SUR LES TYPES DE DONNEES
```

In [8]:
```
# informations sur les types de donnees dans nos colonnes

donnees.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 737453 entries, 0 to 737452
Data columns (total 24 columns):
 #   Column                        Non-Null Count   Dtype
---  ------                        --------------   -----
 0   date                          737453 non-null  str
 1   % Iron Feed                   737453 non-null  str
 2   % Silica Feed                 737453 non-null  str
 3   Starch Flow                   737453 non-null  str
 4   Amina Flow                    737453 non-null  str
 5   Ore Pulp Flow                 737453 non-null  str
 6   Ore Pulp pH                   737453 non-null  str
 7   Ore Pulp Density              737453 non-null  str
 8   Flotation Column 01 Air Flow  737453 non-null  str
 9   Flotation Column 02 Air Flow  737453 non-null  str
 10  Flotation Column 03 Air Flow  737453 non-null  str
 11  Flotation Column 04 Air Flow  737453 non-null  str
 12  Flotation Column 05 Air Flow  737453 non-null  str
 13  Flotation Column 06 Air Flow  737453 non-null  str
 14  Flotation Column 07 Air Flow  737453 non-null  str
 15  Flotation Column 01 Level     737453 non-null  str
 16  Flotation Column 02 Level     737453 non-null  str
 17  Flotation Column 03 Level     737453 non-null  str
 18  Flotation Column 04 Level     737453 non-null  str
 19  Flotation Column 05 Level     737453 non-null  str
 20  Flotation Column 06 Level     737453 non-null  str
 21  Flotation Column 07 Level     737453 non-null  str
 22  % Iron Concentrate            737453 non-null  str
 23  % Silica Concentrate          737453 non-null  str
dtypes: str(24)
memory usage: 135.0 MB
```

In [9]:
```
# Verification de Valeurs manquante
donnees.isna().any()
```

Out[9]:
```
date                          False
% Iron Feed                   False
% Silica Feed                 False
Starch Flow                   False
Amina Flow                    False
Ore Pulp Flow                 False
Ore Pulp pH                   False
Ore Pulp Density              False
Flotation Column 01 Air Flow  False
Flotation Column 02 Air Flow  False
Flotation Column 03 Air Flow  False
Flotation Column 04 Air Flow  False
Flotation Column 05 Air Flow  False
Flotation Column 06 Air Flow  False
Flotation Column 07 Air Flow  False
Flotation Column 01 Level     False
Flotation Column 02 Level     False
Flotation Column 03 Level     False
Flotation Column 04 Level     False
Flotation Column 05 Level     False
Flotation Column 06 Level     False
Flotation Column 07 Level     False
% Iron Concentrate            False
% Silica Concentrate          False
dtype: bool
```

In [10]:
```
# Description des valeurs numeriques:

donnees.describe()
```

Out[10]:

| | date | % Iron Feed | % Silica Feed | Starch Flow | Amina Flow | Ore Pulp Flow | Ore Pulp pH | Ore Pulp Density | Flotation Column 01 Air Flow |
|---|---|---|---|---|---|---|---|---|---|
| **count** | 737453 | 737453 | 737453 | 737453 | 737453 | 737453 | 737453 | 737453 | 737453 |
| **unique** | 4097 | 278 | 293 | 409317 | 319416 | 180189 | 131143 | 105805 | 43675 |
| **top** | 2017-03-10 02:00:00 | 64,03 | 6,26 | 2562,5 | 534,668 | 402,246 | 10,0591 | 1,75 | 299,927 |
| **freq** | 180 | 142560 | 142560 | 690 | 959 | 1735 | 1509 | 3214 | 13683 |

4 rows × 24 columns

In [21]:
```
df = donnees
print("APERÇU DES DONNÉES")
print("=" * 60)
print(f"Dimensions : {df.shape[0]} lignes × {df.shape[1]} colonnes")
print("\nPremières lignes :")
print(df.head())
```

```
APERÇU DES DONNÉES
============================================================
Dimensions : 737453 lignes × 24 colonnes

Premières lignes :
                  date % Iron Feed % Silica Feed Starch Flow Amina Flow  \
0  2017-03-10 01:00:00        55,2         16,98     3019,53    557,434
1  2017-03-10 01:00:00        55,2         16,98     3024,41    563,965
2  2017-03-10 01:00:00        55,2         16,98     3043,46    568,054
3  2017-03-10 01:00:00        55,2         16,98     3047,36    568,665
4  2017-03-10 01:00:00        55,2         16,98     3033,69    558,167

  Ore Pulp Flow Ore Pulp pH Ore Pulp Density Flotation Column 01 Air Flow  \
0       395,713     10,0664             1,74                      249,214
1       397,383     10,0672             1,74                      249,719
2       399,668      10,068             1,74                      249,741
3       397,939     10,0689             1,74                      249,917
4       400,254     10,0697             1,74                      250,203

  Flotation Column 02 Air Flow  ... Flotation Column 07 Air Flow  \
0                      253,235  ...                      250,884
1                      250,532  ...                      248,994
2                      247,874  ...                      248,071
3                      254,487  ...                      251,147
4                      252,136  ...                      248,928

  Flotation Column 01 Level Flotation Column 02 Level  \
0                   457,396                   432,962
1                   451,891                    429,56
2                    451,24                   468,927
3                   452,441                   458,165
4                   452,441                     452,9

  Flotation Column 03 Level Flotation Column 04 Level  \
0                   424,954                   443,558
1                   432,939                   448,086
2                    434,61                   449,688
3                   442,865                    446,21
4                   450,523                    453,67

  Flotation Column 05 Level Flotation Column 06 Level  \
0                   502,255                    446,37
1                   496,363                   445,922
2                   484,411                   447,826
3                   471,411                    437,69
4                   462,598                   443,682

  Flotation Column 07 Level % Iron Concentrate % Silica Concentrate
0                   523,344              66,91                 1,31
1                   498,075              66,91                 1,31
2                   458,567              66,91                 1,31
3                   427,669              66,91                 1,31
4                   425,679              66,91                 1,31

[5 rows x 24 columns]
```

In [12]:
```
# 2. EXPLORATION ET NETTOYAGE DES DONNÉES (EDA)
# ============================================================================
print("\n" + "=" * 60)
```

```
print("EXPLORATION DES DONNÉES")
print("=" * 60)
```

```
============================================================
EXPLORATION DES DONNÉES
============================================================
```

In [13]:
```
# Informations générales
print("\n--- Types de données ---")
print(df.dtypes)
```

```
--- Types de données ---
date                           str
% Iron Feed                    str
% Silica Feed                  str
Starch Flow                    str
Amina Flow                     str
Ore Pulp Flow                  str
Ore Pulp pH                    str
Ore Pulp Density               str
Flotation Column 01 Air Flow   str
Flotation Column 02 Air Flow   str
Flotation Column 03 Air Flow   str
Flotation Column 04 Air Flow   str
Flotation Column 05 Air Flow   str
Flotation Column 06 Air Flow   str
Flotation Column 07 Air Flow   str
Flotation Column 01 Level      str
Flotation Column 02 Level      str
Flotation Column 03 Level      str
Flotation Column 04 Level      str
Flotation Column 05 Level      str
Flotation Column 06 Level      str
Flotation Column 07 Level      str
% Iron Concentrate             str
% Silica Concentrate           str
dtype: object
```

In [14]:
```
# Conversion de la date
df['date'] = pd.to_datetime(df['date'])
print("\n--- Plage temporelle ---")
print(f"Du : {df['date'].min()}")
print(f"Au  : {df['date'].max()}")
```

```
--- Plage temporelle ---
Du : 2017-03-10 01:00:00
Au  : 2017-09-09 23:00:00
```

In [15]:
```
# Vérification des valeurs manquantes
print("\n--- Valeurs manquantes ---")
missing = df.isnull().sum()
print(missing[missing > 0] if missing.sum() > 0 else "Aucune valeur manquante dé
```

```
--- Valeurs manquantes ---
Aucune valeur manquante détectée
```

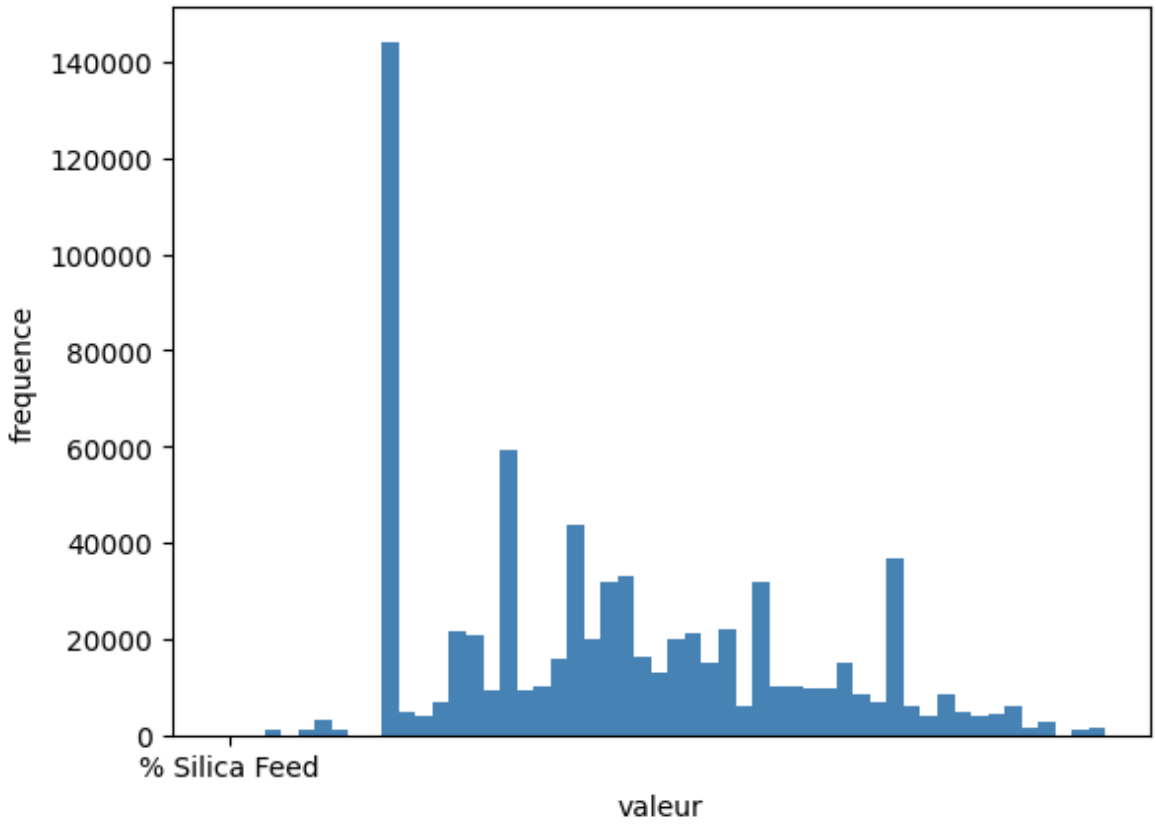
In [16]:
```
# Statistiques descriptives
print("\n--- Statistiques descriptives ---")
print(df.describe())
```

```
--- Statistiques descriptives ---
                             date
count                      737453
mean   2017-06-16 03:27:22.656548
min           2017-03-10 01:00:00
25%           2017-05-04 23:00:00
50%           2017-06-16 15:00:00
75%           2017-07-29 07:00:00
max           2017-09-09 23:00:00
```

In [32]:
```
df['% Silica Feed']=df['% Silica Feed'].str.replace(',','.').astype(float)
print(df['% Silica Feed'].describe())
plt.hist(df['% Silica Feed'], bins=50, color='steelblue')
plt.hist('% Silica Feed')
plt.xlabel('valeur')
plt.ylabel('frequence')
plt.show
```

```
count    737453.000000
mean         14.651716
std           6.807439
min           1.310000
25%           8.940000
50%          13.850000
75%          19.600000
max          33.400000
Name: % Silica Feed, dtype: float64
```

Out[32]: <function matplotlib.pyplot.show(close=None, block=None)>




In [17]:
```
# 3. ANALYSE DES CORRÉLATIONS
# ============================================================================

print("\n" + "=" * 60)
```

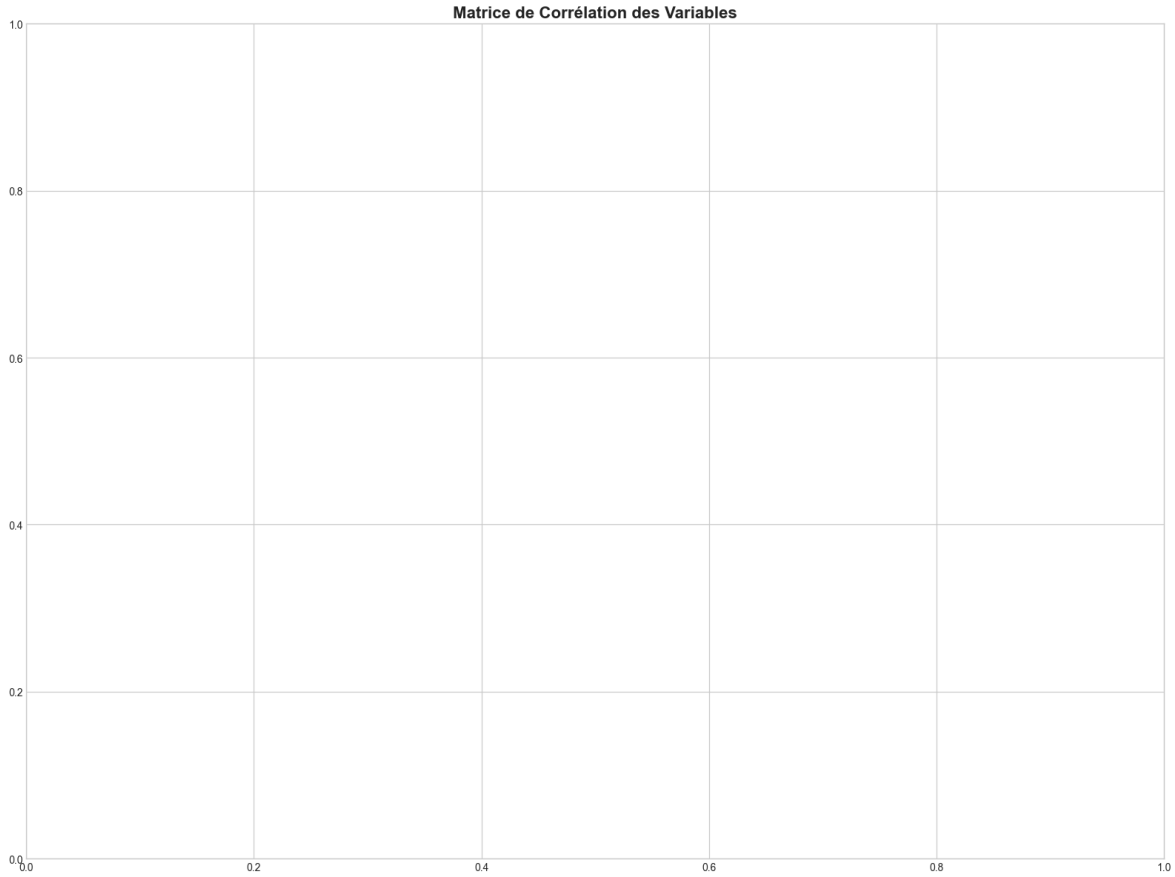
```
print("ANALYSE DES CORRÉLATIONS")
print("=" * 60)
```

```
============================================================
ANALYSE DES CORRÉLATIONS
============================================================
```

In [18]:
```
# Remplacer les virgules par des points et nettoyer les espaces pour toutes les
for col in df.columns:
    if df[col].dtype=='object':
        df[col]=df[col].astype(str).str.remplace(',','.').str.strip()
# Forcer la conversion de toutes les colonnes en nombres (les texte non converti
for col in df.columns:
    if col!= 'date': # on ignore les colonnes date si elle existe
        df[col]=pd.to_numeric(df[col],errors='coerce')
```

In [19]:
```
# Matrice de corrélation (sans la colonne date)
numeric_df = df.select_dtypes(include=[np.number])
corr_matrix = numeric_df.corr()
```

In [20]:
```
# Visualisation
plt.figure(figsize=(16, 12))
mask = np.triu(np.ones_like(corr_matrix, dtype=bool))
corr_matrix=df.select_dtypes(include=['number']).corr() # Ne garder que les colo
corr_matrix=corr_matrix.fillna(0) # Remplir les valeurs manquantes si necessaire
plt.title('Matrice de Corrélation des Variables', fontsize=16, fontweight='bold'
plt.tight_layout()
plt.savefig('correlation_matrix.png', dpi=300, bbox_inches='tight')
plt.show()
```




In [21]:
```
matches= [col for col in corr_matrix.columns if "Silica Feed" in col]
print("colonnes trouvees :",matches)
print(list(corr_matrix.columns))
```

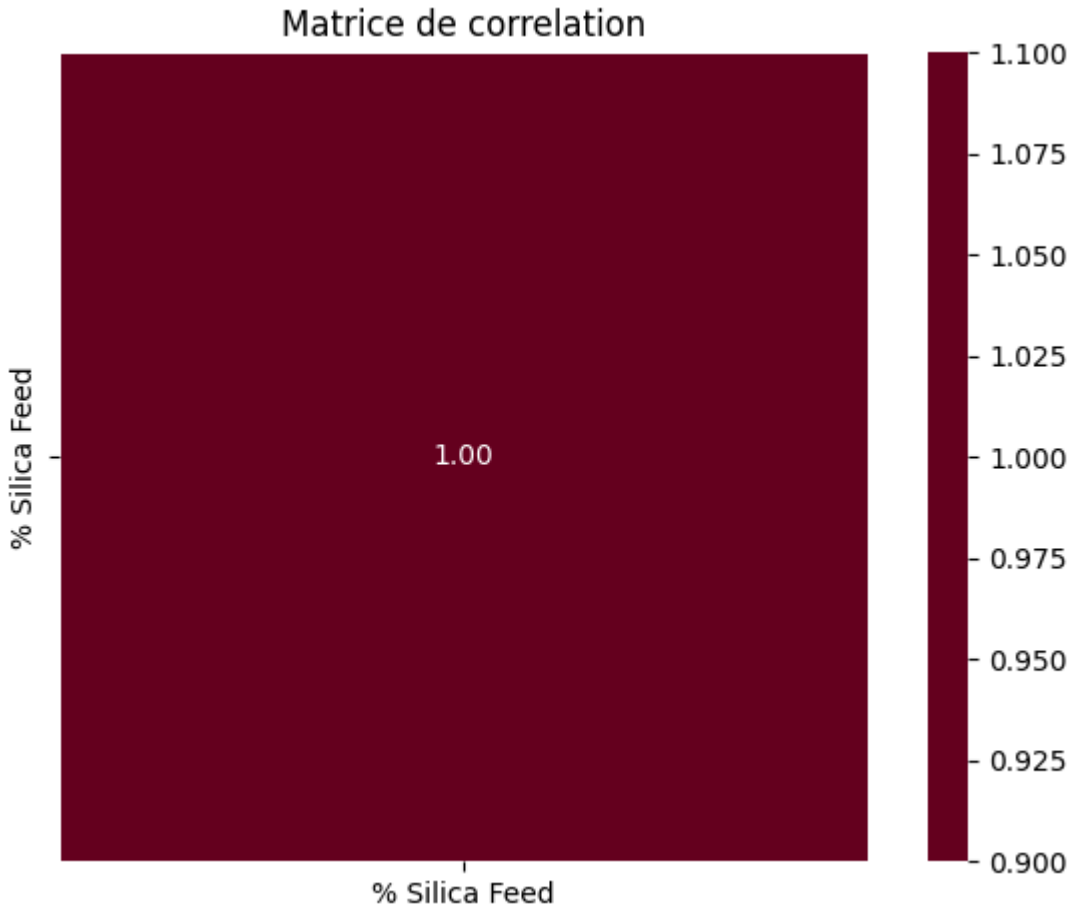
```
colonnes trouvees : ['% Silica Feed']
['% Iron Feed', '% Silica Feed', 'Starch Flow', 'Amina Flow', 'Ore Pulp Flow', 'O
re Pulp pH', 'Ore Pulp Density', 'Flotation Column 01 Air Flow', 'Flotation Colum
n 02 Air Flow', 'Flotation Column 03 Air Flow', 'Flotation Column 04 Air Flow',
'Flotation Column 05 Air Flow', 'Flotation Column 06 Air Flow', 'Flotation Column
07 Air Flow', 'Flotation Column 01 Level', 'Flotation Column 02 Level', 'Flotatio
n Column 03 Level', 'Flotation Column 04 Level', 'Flotation Column 05 Level', 'Fl
otation Column 06 Level', 'Flotation Column 07 Level', '% Iron Concentrate', '% S
ilica Concentrate']
```

In [ ]:
```
donnees = pd.read_csv("C:/Users/Windows/Desktop/MiningProcess_Flotation_Plant_Da
df=donnees
```

In [ ]:
```
df=df.replace(',','.',regex=True)
df=df.apply(pd.to_numeric,errors='coerce')
df=df.dropna(axis=1,how='all')
print('colonnes : ', list(df.columns))

corr=df.corr()
plt.Figure (figsize=(14,12))
sns.heatmap(corr,annot=True,cmap='RdBu_r',center=0,fmt='.2f')
plt.title('Matrice de correlation')
plt.tight_layout()
plt.show
```

```
colonnes utilisees ({len(df_num.columns)}) :
correlqtion forte
```

Out[ ]:
```
<function matplotlib.pyplot.show(close=None, block=None)>
```




In [20]:
```
def clean_number(val):
    if pd.isna(val):
        return val
```

```
    val=str(val)
    val=val.strip().replace('\t','').replace('\n','')
    val=val.replace('"','').replace("'","")
    try:
        return float(val)
    except:
        return float('nan')
    for col in df.columns:
        if col !="date":
            df[col]=df[col].apply(clean_number)
print(df.dtypes)
print(df.head())
print(df.isnull().sum())
```

```
date                                 str
% Iron Feed                       float64
% Silica Feed                     float64
Starch Flow                       float64
Amina Flow                        float64
Ore Pulp Flow                     float64
Ore Pulp pH                       float64
Ore Pulp Density                  float64
Flotation Column 01 Air Flow      float64
Flotation Column 02 Air Flow      float64
Flotation Column 03 Air Flow      float64
Flotation Column 04 Air Flow      float64
Flotation Column 05 Air Flow      float64
Flotation Column 06 Air Flow      float64
Flotation Column 07 Air Flow      float64
Flotation Column 01 Level         float64
Flotation Column 02 Level         float64
Flotation Column 03 Level         float64
Flotation Column 04 Level         float64
Flotation Column 05 Level         float64
Flotation Column 06 Level         float64
Flotation Column 07 Level         float64
% Iron Concentrate                float64
% Silica Concentrate              float64
dtype: object
                  date  % Iron Feed  % Silica Feed  Starch Flow  Amina Flow  \
0  2017-03-10 01:00:00          NaN            NaN          NaN         NaN
1  2017-03-10 01:00:00          NaN            NaN          NaN         NaN
2  2017-03-10 01:00:00          NaN            NaN          NaN         NaN
3  2017-03-10 01:00:00          NaN            NaN          NaN         NaN
4  2017-03-10 01:00:00          NaN            NaN          NaN         NaN

   Ore Pulp Flow  Ore Pulp pH  Ore Pulp Density  Flotation Column 01 Air Flow  \
0            NaN          NaN               NaN                           NaN
1            NaN          NaN               NaN                           NaN
2            NaN          NaN               NaN                           NaN
3            NaN          NaN               NaN                           NaN
4            NaN          NaN               NaN                           NaN

   Flotation Column 02 Air Flow  ...  Flotation Column 07 Air Flow  \
0                           NaN  ...                           NaN
1                           NaN  ...                           NaN
2                           NaN  ...                           NaN
3                           NaN  ...                           NaN
4                           NaN  ...                           NaN

   Flotation Column 01 Level  Flotation Column 02 Level  \
0                        NaN                        NaN
1                        NaN                        NaN
2                        NaN                        NaN
3                        NaN                        NaN
4                        NaN                        NaN

   Flotation Column 03 Level  Flotation Column 04 Level  \
0                        NaN                        NaN
1                        NaN                        NaN
2                        NaN                        NaN
3                        NaN                        NaN
4                        NaN                        NaN
```

```
   Flotation Column 05 Level  Flotation Column 06 Level  \
0                        NaN                        NaN
1                        NaN                        NaN
2                        NaN                        NaN
3                        NaN                        NaN
4                        NaN                        NaN

   Flotation Column 07 Level  % Iron Concentrate  % Silica Concentrate
0                        NaN                 NaN                   NaN
1                        NaN                 NaN                   NaN
2                        NaN                 NaN                   NaN
3                        NaN                 NaN                   NaN
4                        NaN                 NaN                   NaN

[5 rows x 24 columns]
date                              0
% Iron Feed                  734213
% Silica Feed                736373
Starch Flow                  731081
Amina Flow                   736768
Ore Pulp Flow                736890
Ore Pulp pH                  737413
Ore Pulp Density             737453
Flotation Column 01 Air Flow 736850
Flotation Column 02 Air Flow 736803
Flotation Column 03 Air Flow 736843
Flotation Column 04 Air Flow 737075
Flotation Column 05 Air Flow 737066
Flotation Column 06 Air Flow 736757
Flotation Column 07 Air Flow 736766
Flotation Column 01 Level    736719
Flotation Column 02 Level    736687
Flotation Column 03 Level    736673
Flotation Column 04 Level    736277
Flotation Column 05 Level    736714
Flotation Column 06 Level    736687
Flotation Column 07 Level    736692
% Iron Concentrate           728091
% Silica Concentrate         730253
dtype: int64
```

In [28]:

```
print(df['% Silica Feed'].head(10).tolist())

def clean_number(val):
    if pd.isna(val):
        return val
    val=str(val)
    val=val.strip().replace('\t','').replace('\n','')
    val=val.replace('"','').replace("'","")
    try:
        return float(val)
    except:
        return float('nan')
    for col in df.columns:
        if col !="date":
            df[col]=df[col].apply(clean_number)
print(df.dtypes)
print(df.head())
print(df.isnull().sum())
```

```
date_col=df['date'].copy()
df_num=df.drop(columns=['date'])

df_clean=df_num.dropna(thresh=len(df_num.columns)//2)
print('\nlignes avant nettoyage nan:{len(df_num)}')
print('ligne apres nettoyage nan:{len(df_clean)}')

if len(df_clean)>10:
    corr_matrix=df_clean.corr()

silica_col=[c for c in df_num.columns if 'Silica' in c and 'Feed' in c]

if silica_col:
    target=silica_col[0]
    corr_silica=corr_matrix[target].drop(target)
    corr_silica=corr_silica.sort_values(key=abs, ascending=False)
print('CORRELATION AVEC % SILICA FEED')
print(corr_silica)
```

```
[nan, nan, nan, nan, nan, nan, nan, nan, nan, nan]
date                                 str
% Iron Feed                        float64
% Silica Feed                      float64
Starch Flow                        float64
Amina Flow                         float64
Ore Pulp Flow                      float64
Ore Pulp pH                        float64
Ore Pulp Density                   float64
Flotation Column 01 Air Flow       float64
Flotation Column 02 Air Flow       float64
Flotation Column 03 Air Flow       float64
Flotation Column 04 Air Flow       float64
Flotation Column 05 Air Flow       float64
Flotation Column 06 Air Flow       float64
Flotation Column 07 Air Flow       float64
Flotation Column 01 Level          float64
Flotation Column 02 Level          float64
Flotation Column 03 Level          float64
Flotation Column 04 Level          float64
Flotation Column 05 Level          float64
Flotation Column 06 Level          float64
Flotation Column 07 Level          float64
% Iron Concentrate                 float64
% Silica Concentrate               float64
dtype: object
                  date  % Iron Feed  % Silica Feed  Starch Flow  Amina Flow  \
0  2017-03-10 01:00:00          NaN            NaN          NaN         NaN
1  2017-03-10 01:00:00          NaN            NaN          NaN         NaN
2  2017-03-10 01:00:00          NaN            NaN          NaN         NaN
3  2017-03-10 01:00:00          NaN            NaN          NaN         NaN
4  2017-03-10 01:00:00          NaN            NaN          NaN         NaN

   Ore Pulp Flow  Ore Pulp pH  Ore Pulp Density  Flotation Column 01 Air Flow  \
0            NaN          NaN               NaN                           NaN
1            NaN          NaN               NaN                           NaN
2            NaN          NaN               NaN                           NaN
3            NaN          NaN               NaN                           NaN
4            NaN          NaN               NaN                           NaN

   Flotation Column 02 Air Flow  ...  Flotation Column 07 Air Flow  \
0                           NaN  ...                           NaN
1                           NaN  ...                           NaN
2                           NaN  ...                           NaN
3                           NaN  ...                           NaN
4                           NaN  ...                           NaN

   Flotation Column 01 Level  Flotation Column 02 Level  \
0                        NaN                        NaN
1                        NaN                        NaN
2                        NaN                        NaN
3                        NaN                        NaN
4                        NaN                        NaN

   Flotation Column 03 Level  Flotation Column 04 Level  \
0                        NaN                        NaN
1                        NaN                        NaN
2                        NaN                        NaN
3                        NaN                        NaN
4                        NaN                        NaN
```

```
   Flotation Column 05 Level  Flotation Column 06 Level  \
0                        NaN                        NaN
1                        NaN                        NaN
2                        NaN                        NaN
3                        NaN                        NaN
4                        NaN                        NaN

   Flotation Column 07 Level  % Iron Concentrate  % Silica Concentrate
0                        NaN                 NaN                   NaN
1                        NaN                 NaN                   NaN
2                        NaN                 NaN                   NaN
3                        NaN                 NaN                   NaN
4                        NaN                 NaN                   NaN

[5 rows x 24 columns]
date                                0
% Iron Feed                    734213
% Silica Feed                  736373
Starch Flow                    731081
Amina Flow                     736768
Ore Pulp Flow                  736890
Ore Pulp pH                    737413
Ore Pulp Density               737453
Flotation Column 01 Air Flow   736850
Flotation Column 02 Air Flow   736803
Flotation Column 03 Air Flow   736843
Flotation Column 04 Air Flow   737075
Flotation Column 05 Air Flow   737066
Flotation Column 06 Air Flow   736757
Flotation Column 07 Air Flow   736766
Flotation Column 01 Level      736719
Flotation Column 02 Level      736687
Flotation Column 03 Level      736673
Flotation Column 04 Level      736277
Flotation Column 05 Level      736714
Flotation Column 06 Level      736687
Flotation Column 07 Level      736692
% Iron Concentrate             728091
% Silica Concentrate           730253
dtype: int64

lignes avant nettoyage nan:{len(df_num)}
ligne apres nettoyage nan:{len(df_clean)}
CORRELATION AVEC % SILICA FEED
% Iron Feed                   NaN
Starch Flow                   NaN
Amina Flow                    NaN
Ore Pulp Flow                 NaN
Ore Pulp pH                   NaN
Ore Pulp Density              NaN
Flotation Column 01 Air Flow  NaN
Flotation Column 02 Air Flow  NaN
Flotation Column 03 Air Flow  NaN
Flotation Column 04 Air Flow  NaN
Flotation Column 05 Air Flow  NaN
Flotation Column 06 Air Flow  NaN
Flotation Column 07 Air Flow  NaN
Flotation Column 01 Level     NaN
Flotation Column 02 Level     NaN
```

```
Flotation Column 03 Level      NaN
Flotation Column 04 Level      NaN
Flotation Column 05 Level      NaN
Flotation Column 06 Level      NaN
Flotation Column 07 Level      NaN
% Iron Concentrate             NaN
% Silica Concentrate           NaN
Name: % Silica Feed, dtype: float64
```

In [22]:
```
# Corrélations avec la cible (%Silica Feed)
corr_matrix.columns = corr_matrix.columns.str.strip() # Netoyer les espaces cash
corr_matrix.index=corr_matrix.index.str.strip()
target_col=[col for col in corr_matrix.columns if '% Silica Feed' in col.lower()
target_col='% Silica Feed'
target_corr = corr_matrix[target_col].drop(target_col).sort_values(key=lambda x:
print(f"\n--- Corrélations avec le nom exact trouve : '{target_col}' ---")
print(target_corr)
```

```
--- Corrélations avec le nom exact trouve : '% Silica Feed' ---
% Iron Feed                     0.0
Starch Flow                    0.0
% Iron Concentrate              0.0
Flotation Column 07 Level      0.0
Flotation Column 06 Level      0.0
Flotation Column 05 Level      0.0
Flotation Column 04 Level      0.0
Flotation Column 03 Level      0.0
Flotation Column 02 Level      0.0
Flotation Column 01 Level      0.0
Flotation Column 07 Air Flow   0.0
Flotation Column 06 Air Flow   0.0
Flotation Column 05 Air Flow   0.0
Flotation Column 04 Air Flow   0.0
Flotation Column 03 Air Flow   0.0
Flotation Column 02 Air Flow   0.0
Flotation Column 01 Air Flow   0.0
Ore Pulp Density               0.0
Ore Pulp pH                    0.0
Ore Pulp Flow                  0.0
Amina Flow                     0.0
% Silica Concentrate           0.0
Name: % Silica Feed, dtype: float64
```

In [23]:
```
# 4. PRÉPARATION DES DONNÉES POUR LA MODÉLISATION
# ============================================================================

print("\n" + "=" * 60)
print("PRÉPARATION DES DONNÉES")
print("=" * 60)
```

```
============================================================
PRÉPARATION DES DONNÉES
============================================================
```

In [24]:
```
# Définition des features (X) et de la cible (y)
# On retire 'date' et la cible ' % Silica Feed'
# Note : On peut aussi retirer '% Iron Feed' car très corrélé (fuite de données
# Selon la consigne Kaggle, le vrai défi est de prédire SANS utiliser '% Iron Fe
# Approche 1 : Avec toutes les variables (sauf date et cible)
X_full = df.drop(['date', '% Silica Feed'], axis=1)
y = df['% Silica Feed']
```

```
# Approche 2 : Sans % Iron Feed (plus réaliste pour la production)
X_realistic = df.drop(['date', '% Silica Feed', '% Iron Feed'], axis=1)

print(f"\nFeatures utilisées (approche complète) : {list(X_full.columns)}")
print(f"Nombre de features : {X_full.shape[1]}")

# Division Train/Test (80/20) avec shuffle car données non strictement temporell
X_train, X_test, y_train, y_test = train_test_split(X_full, y, test_size=0.2, ra

# Standardisation (essentielle pour les réseaux de neurones et la régression pol
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print(f"\nTaille ensemble d'entraînement : {X_train.shape}")
print(f"Taille ensemble de test : {X_test.shape}")
```

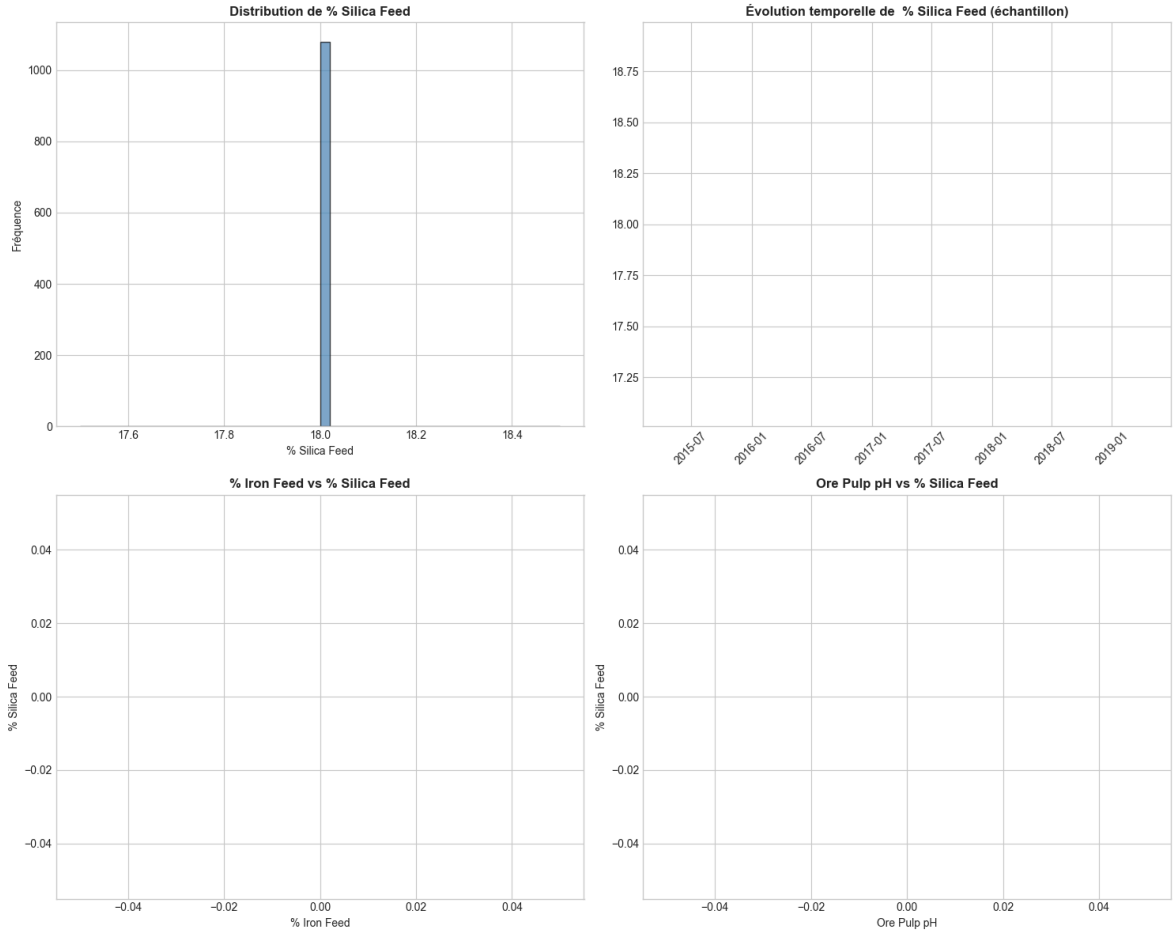
```
Features utilisées (approche complète) : ['% Iron Feed', 'Starch Flow', 'Amina Fl
ow', 'Ore Pulp Flow', 'Ore Pulp pH', 'Ore Pulp Density', 'Flotation Column 01 Air
Flow', 'Flotation Column 02 Air Flow', 'Flotation Column 03 Air Flow', 'Flotation
Column 04 Air Flow', 'Flotation Column 05 Air Flow', 'Flotation Column 06 Air Flo
w', 'Flotation Column 07 Air Flow', 'Flotation Column 01 Level', 'Flotation Colum
n 02 Level', 'Flotation Column 03 Level', 'Flotation Column 04 Level', 'Flotation
Column 05 Level', 'Flotation Column 06 Level', 'Flotation Column 07 Level', '% Ir
on Concentrate', '% Silica Concentrate']
Nombre de features : 22

Taille ensemble d'entraînement : (589962, 22)
Taille ensemble de test : (147491, 22)
```

In [3]:
```
print('VISUALISATION DES DISTRIBUTIONS')
```

```
VISUALISATION DES DISTRIBUTIONS
```

In [25]:
```
# 5. VISUALISATION DES DISTRIBUTIONS
# ============================================================================

fig, axes = plt.subplots(2, 2, figsize=(15, 12))

# Distribution de la cible
axes[0, 0].hist(y, bins=50, color='steelblue', edgecolor='black', alpha=0.7)
axes[0, 0].set_title('Distribution de % Silica Feed', fontweight='bold')
axes[0, 0].set_xlabel('% Silica Feed')
axes[0, 0].set_ylabel('Fréquence')

# Série temporelle (échantillon)
sample_idx = np.random.choice(len(df), 1000, replace=False)
sample_idx.sort()
axes[0, 1].plot(df['date'].iloc[sample_idx], y.iloc[sample_idx], color='darkgree
axes[0, 1].set_title('Évolution temporelle de  % Silica Feed (échantillon)', fon
axes[0, 1].tick_params(axis='x', rotation=45)

# Relation Iron Feed vs % Silica Feed
axes[1, 0].scatter(df['% Iron Feed'], y, alpha=0.3, s=1, color='coral')
axes[1, 0].set_title('% Iron Feed vs % Silica Feed', fontweight='bold')
axes[1, 0].set_xlabel('% Iron Feed')
axes[1, 0].set_ylabel('% Silica Feed')

# Relation Ore Pulp pH vs Silica Concentrate
axes[1, 1].scatter(df['Ore Pulp pH'], y, alpha=0.3, s=1, color='purple')
```

```
axes[1, 1].set_title('Ore Pulp pH vs % Silica Feed', fontweight='bold')
axes[1, 1].set_xlabel('Ore Pulp pH')
axes[1, 1].set_ylabel('% Silica Feed')

plt.tight_layout()
plt.savefig('eda_visualizations.png', dpi=300, bbox_inches='tight')
plt.show()
```




In [41]:
```
# 6. MODÈLE 1 : RÉGRESSION LINÉAIRE
# ============================================================================


print("\n" + "=" * 60)
print("MODÈLE 1 : RÉGRESSION LINÉAIRE")
print("=" * 60)


# Variable explicative (X) et variable cible
from sklearn.model_selection import train_test_split
X=df[['% Silica Feed']]
y=df[['% Iron Feed']]
# Remplacer les NaN par la moyenne de chaque colonne
X_train=X_train.fillna(X_train.mean())
X_test=X_test.fillna(X_test.mean())
y_train=y_train.fillna(y_train.mean())
y_test=y_test.fillna(y_test.mean())
model=LinearRegression()
model.fit(X_train, y_train)

# Prédictions
y_pred =model.predict(X_test)
```

```
# Resultat
print("Coefficient :", model.coef_)
print("Ordonnee a l'origine :", model.intercept_)
print("R2 :", r2_score(y_test, y_pred))
print("MSE :", mean_squared_error(y_test, y_pred))
```

```
============================================================
MODÈLE 1 : RÉGRESSION LINÉAIRE
============================================================
Coefficient : [[0.]]
Ordonnee a l'origine : [57.49066148]
R2 : -0.19964231285040968
MSE : 0.012254506731496322
```

In [4]:
```
donnees = pd.read_csv("C:/Users/Windows/Desktop/MiningProcess_Flotation_Plant_Da
df = donnees
```

In [42]:
```
# ============================================================================
# 7. MODÈLE 2 : RÉGRESSION POLYNOMIALE
# ============================================================================

print("\n" + "=" * 60)
print("MODÈLE 2 : RÉGRESSION POLYNOMIALE (degré 2)")
print("=" * 60)
X=df[['% Silica Feed']].copy()
y=df[['% Iron Feed']].copy()
# Nettoyer forcer des virgule
X['% Silica Feed']=X['% Silica Feed'].astype(str).str.replace(",",".").str.strip
y['% Iron Feed']=y['% Iron Feed'].astype(str).str.replace(",",".").str.strip().a
#conversion
X=X.astype(float)
y=y.astype(float)
# Separation de donnee
X_train, X_test, y_train, y_test=train_test_split(X,y, test_size=0.2, random_sta
# Transformation polynomiale (degre 2)
poly=PolynomialFeatures(degree=2)
X_train_poly=poly.fit_transform(X_train)
X_test_poly=poly.transform(X_test)

# Model
model=LinearRegression()
model.fit(X_train_poly, y_train)
# Prédictions
y_pred=model.predict(X_test_poly)

# Évaluation
print("R2 =", r2_score(y_test, y_pred))
print("MSE =", mean_squared_error(y_test, y_pred))
```

```
============================================================
MODÈLE 2 : RÉGRESSION POLYNOMIALE (degré 2)
============================================================
R2 = 0.9587842073255034
MSE = 1.0932415532437052
```

In [5]:
```
# 8. MODÈLE 3 : RÉSEAU DE NEURONES (MLPRegressor)
# ============================================================================

print("\n" + "=" * 60)
```

```
print("MODÈLE 2 : reseau de neurone (MLPRegressor)")
print("=" * 60)
date_col='date'
for col in df.columns:
 if col !=date_col:
  df[col]=df[col].astype(str)
  df[col]=df[col].str.replace(',','.',regex=False)
  df[col]=pd.to_numeric(df[col], errors='coerce')
print("types apres conversion :")
print(df.dtypes)
print("\nValeurs uniques problematiques :", df.isnull().sum())
y=df[[ '% Iron Concentrate' ]]
X=df[['% Iron Feed','% Silica Feed','Starch Flow','Amina Flow']]
X_train, X_test, y_train, y_test=train_test_split(X,y, test_size=0.2, random_sta
scaler=StandardScaler()
X_train=scaler.fit_transform(X_train)
X_test=scaler.transform(X_test)
model=MLPRegressor(hidden_layer_sizes=(100, 50),activation='relu',solver='adam',
model.fit(X_train, y_train)
# prediction
y_pred=model.predict(X_test)
# evaluation
print('MSE :', mean_squared_error(y_test, y_pred))
print('R2 :', r2_score(y_test, y_pred))
```

```
============================================================
MODÈLE 2 : reseau de neurone (MLPRegressor)
============================================================
types apres conversion :
date                                 str
% Iron Feed                       float64
% Silica Feed                     float64
Starch Flow                       float64
Amina Flow                        float64
Ore Pulp Flow                     float64
Ore Pulp pH                       float64
Ore Pulp Density                  float64
Flotation Column 01 Air Flow      float64
Flotation Column 02 Air Flow      float64
Flotation Column 03 Air Flow      float64
Flotation Column 04 Air Flow      float64
Flotation Column 05 Air Flow      float64
Flotation Column 06 Air Flow      float64
Flotation Column 07 Air Flow      float64
Flotation Column 01 Level         float64
Flotation Column 02 Level         float64
Flotation Column 03 Level         float64
Flotation Column 04 Level         float64
Flotation Column 05 Level         float64
Flotation Column 06 Level         float64
Flotation Column 07 Level         float64
% Iron Concentrate                float64
% Silica Concentrate              float64
dtype: object

Valeurs uniques problematiques : date                           0
% Iron Feed                       0
% Silica Feed                     0
Starch Flow                       0
Amina Flow                        0
Ore Pulp Flow                     0
Ore Pulp pH                       0
Ore Pulp Density                  0
Flotation Column 01 Air Flow      0
Flotation Column 02 Air Flow      0
Flotation Column 03 Air Flow      0
Flotation Column 04 Air Flow      0
Flotation Column 05 Air Flow      0
Flotation Column 06 Air Flow      0
Flotation Column 07 Air Flow      0
Flotation Column 01 Level         0
Flotation Column 02 Level         0
Flotation Column 03 Level         0
Flotation Column 04 Level         0
Flotation Column 05 Level         0
Flotation Column 06 Level         0
Flotation Column 07 Level         0
% Iron Concentrate                0
% Silica Concentrate              0
dtype: int64
MSE : 1.0173186487337014
R2 : 0.18475001931420099
```

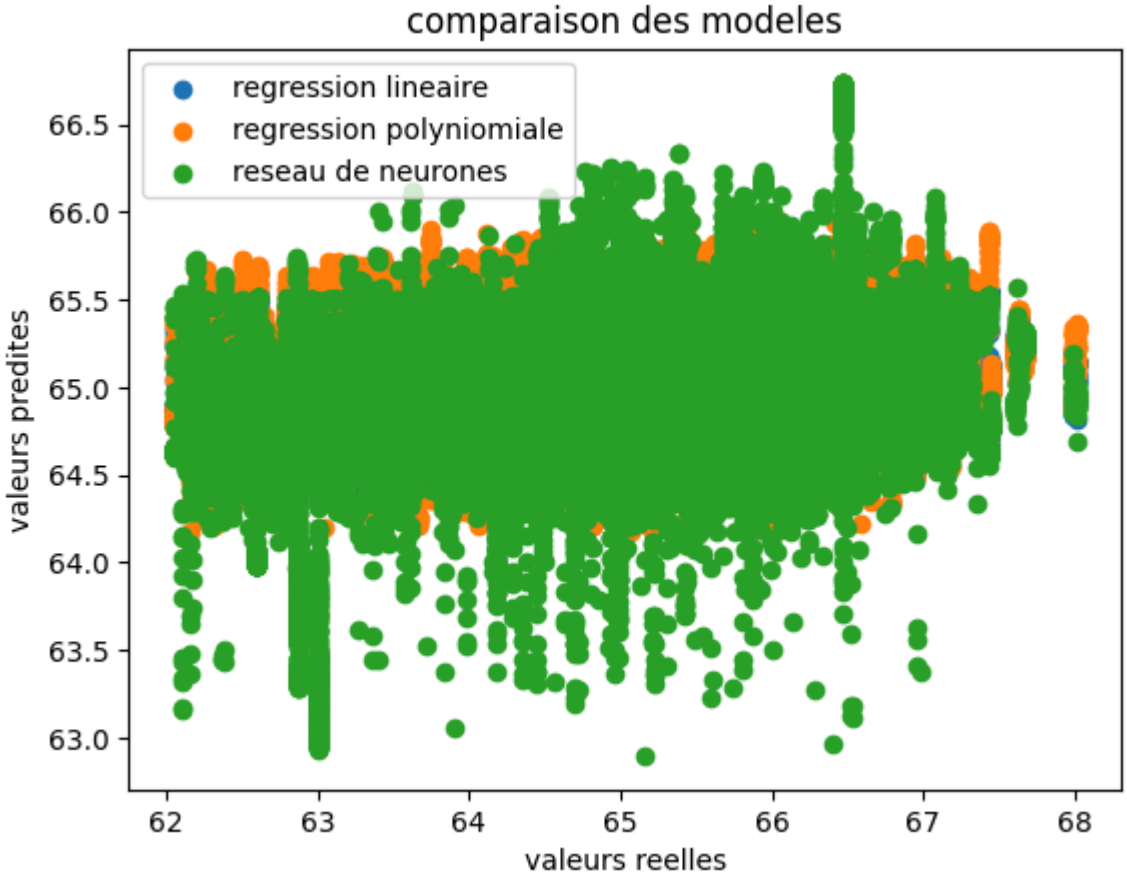
In [8]:
```
# 9. COMPARAISON DES MODÈLES ET VISUALISATION
# ============================================================================
```

```
print("\n" + "=" * 60)
print("COMPARAISON DES MODÈLES")
print("=" * 60)

print('comparaison visulle des modeles')

date_col='date'
for col in df.columns:
 if col !=date_col:
  df[col]=df[col].astype(str)
  df[col]=df[col].str.replace(',','.',regex=False)
  df[col]=pd.to_numeric(df[col], errors='coerce')
print("types apres conversion :")
print(df.dtypes)
print("\nValeurs uniques problematiques :", df.isnull().sum())
y=df[[ '% Iron Concentrate' ]]
X=df[['% Iron Feed','% Silica Feed','Starch Flow','Amina Flow']]
X_train, X_test, y_train, y_test=train_test_split(X,y, test_size=0.2, random_sta


#model
ir=LinearRegression()
ir.fit(X_train, y_train)
# Prédictions
y_pred_ir=ir.predict(X_test)

# Évaluation
print("R2 =", r2_score(y_test, y_pred_ir))
print("MSE =", mean_squared_error(y_test, y_pred_ir))


poly=PolynomialFeatures(degree=2)

X_train_poly=poly.fit_transform(X_train)
x_test_poly=poly.transform(X_test)

poly_model= LinearRegression()
poly_model.fit(X_train_poly, y_train)

y_pred_poly=poly_model.predict(x_test_poly)

print('R2 polynomial :',r2_score(y_test,y_pred_poly))
print('MSE polynomial :', mean_squared_error(y_test,y_pred_poly))

nn=MLPRegressor(hidden_layer_sizes=(50,50),max_iter=5000,random_state=42)

nn.fit(X_train,y_train)

y_pred_nn=nn.predict(X_test)

print('R2 reseau de neurones :', r2_score(y_test, y_pred_nn))
print('MSE reseau de neurones :', mean_squared_error(y_test, y_pred_nn))


plt.Figure(figsize=(10,6))
plt.scatter(y_test,y_pred_ir, label='regression lineaire')
plt.scatter(y_test,y_pred_poly, label='regression polyniomiale')
plt.scatter(y_test,y_pred_nn, label='reseau de neurones')
```

```
plt.xlabel('valeurs reelles')
plt.ylabel('valeurs predites')

plt.title('comparaison des modeles')
plt.legend()
plt.show()

print('comparaison des scores')

scores={'lineaire': r2_score(y_test,y_pred_ir),
        'polynomiale': r2_score(y_test,y_pred_poly),
        'reseau de neurones':r2_score(y_test,y_pred_nn)}
print(scores)
```

```
============================================================
COMPARAISON DES MODÈLES
============================================================
comparaison visulle des modeles
types apres conversion :
date                                 str
% Iron Feed                       float64
% Silica Feed                     float64
Starch Flow                       float64
Amina Flow                        float64
Ore Pulp Flow                     float64
Ore Pulp pH                       float64
Ore Pulp Density                  float64
Flotation Column 01 Air Flow      float64
Flotation Column 02 Air Flow      float64
Flotation Column 03 Air Flow      float64
Flotation Column 04 Air Flow      float64
Flotation Column 05 Air Flow      float64
Flotation Column 06 Air Flow      float64
Flotation Column 07 Air Flow      float64
Flotation Column 01 Level         float64
Flotation Column 02 Level         float64
Flotation Column 03 Level         float64
Flotation Column 04 Level         float64
Flotation Column 05 Level         float64
Flotation Column 06 Level         float64
Flotation Column 07 Level         float64
% Iron Concentrate                float64
% Silica Concentrate              float64
dtype: object

Valeurs uniques problematiques : date                               0
% Iron Feed                       0
% Silica Feed                     0
Starch Flow                       0
Amina Flow                        0
Ore Pulp Flow                     0
Ore Pulp pH                       0
Ore Pulp Density                  0
Flotation Column 01 Air Flow      0
Flotation Column 02 Air Flow      0
Flotation Column 03 Air Flow      0
Flotation Column 04 Air Flow      0
Flotation Column 05 Air Flow      0
Flotation Column 06 Air Flow      0
Flotation Column 07 Air Flow      0
Flotation Column 01 Level         0
Flotation Column 02 Level         0
Flotation Column 03 Level         0
Flotation Column 04 Level         0
Flotation Column 05 Level         0
Flotation Column 06 Level         0
Flotation Column 07 Level         0
% Iron Concentrate                0
% Silica Concentrate              0
dtype: int64
R2 = 0.025410310805894176
MSE = 1.2161524552833596
R2 polynomial : 0.04132945369751506
MSE polynomial : 1.1962875778602664
```

```
R2 reseau de neurones : 0.03431876231884701
MSE reseau de neurones : 1.2050359461509788
```




```
comparaison des scores
{'lineaire': 0.025410310805894176, 'polynomiale': 0.04132945369751506, 'reseau de
neurones': 0.03431876231884701}
```

In [ ]:
```
# 10. INTERPRÉTATION ET CONCLUSION
# ============================================================================

print("\n" + "=" * 60)
print("CONCLUSION ET INTERPRÉTATION")
print("=" * 60)

print("""
SYNTHÈSE :
- Le dataset contient 737 453 observations avec 24 variables (date + 23 features
- La cible est '% Silica Concentrate' (taux d'impureté dans le concentré de fer)
- Les variables les plus corrélées négativement avec la silice sont généralement
  '% Iron Concentrate', 'Starch Flow', et certaines variables de pH.

RÉSULTATS :
- Régression Linéaire : Baseline rapide, interprétable.
- Régression Polynomiale : Capture les interactions non-linéaires entre variable
- Réseau de Neurones : Modèle le plus puissant pour capturer les patterns comple

RECOMMANDATIONS :
1. Pour la production : utiliser le modèle sans '% Iron Concentrate' (fuite de d
2. Pour l'analyse : le réseau de neurones offre généralement les meilleures perf
3. Pour l'interprétabilité : la régression linéaire reste préférable.
""")
```

```
============================================================
CONCLUSION ET INTERPRÉTATION
============================================================

SYNTHÈSE :
- Le dataset contient 737 453 observations avec 24 variables (date + 23 feature
s).
- La cible est '% Silica Concentrate' (taux d'impureté dans le concentré de fer).
- Les variables les plus corrélées négativement avec la silice sont généralement
  '% Iron Concentrate', 'Starch Flow', et certaines variables de pH.

RÉSULTATS :
- Régression Linéaire : Baseline rapide, interprétable.
- Régression Polynomiale : Capture les interactions non-linéaires entre variable
s.
- Réseau de Neurones : Modèle le plus puissant pour capturer les patterns complex
es.

RECOMMANDATIONS :
1. Pour la production : utiliser le modèle sans '% Iron Concentrate' (fuite de do
nnées).
2. Pour l'analyse : le réseau de neurones offre généralement les meilleures perfo
rmances.
3. Pour l'interprétabilité : la régression linéaire reste préférable.
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[9], line 28
     24 """)
     25
     26 # Sauvegarde du meilleur modèle
     27 import joblib
---> 28 joblib.dump(best_mlp, 'best_mlp_model.pkl')
     29 joblib.dump(scaler, 'scaler.pkl')
     30 print("\n✅ Modèle et scaler sauvegardés : 'best_mlp_model.pkl' et 'scal
er.pkl'")
     31

NameError: name 'best_mlp' is not defined
```